CPSC 481
Artificial Intelligence

Chuck Chekuri

cchekuri@fullerton.edu

LinkedIn

Group

Syllabus

Week 8: Learning from Examples

- Introduction to Chapter 19 (Learning from Examples).

Bridging the gap between "Maybe" and "Improvement."
Moving from fixed probabilities to data-driven discovery.

Recap: Bayesian Reasoning

- Conditional Probability: P(a|b) = P(a ^ b) / P(b).
- Bayes' Rule: P(cause|effect) = P(effect|cause) * P(cause) / P(effect).
- Scaling issues with the Full Joint Distribution.
- The need for independence to handle real-world complexity.

The Naive Bayes Model

- Assumes effects are conditionally independent given the cause.
- Formula: $P(Cause \mid E1, ..., En) = \alpha * P(Cause) * \Pi P(Ei \mid Cause)$.
- Drastic reduction in parameters from $O(2^n)$ to $O(n)$.
- The most common probabilistic model in AI history.

Why is it "Naive"?

- It assumes features (symptoms) don't influence each other.
- Reality: Toothache and bleeding gums often co-occur.
- Even if the assumption is false, the ranking is often correct.
- Surprisingly robust in many noisy environments.

Example: Text Classification (Spam Filter)

- Cause: Email is Spam or Not Spam.
- Effects: Presence of individual words (e.g., "Winner", "Meeting").
- We calculate the likelihood based on word counts.
- Result: A high-speed, scalable classification tool.

Global Independence vs. Conditional Independence

- Global: $P(a,b) = P(a)P(b)$ (e.g., Weather and Dental Health).
- Conditional: $P(a,b|c) = P(a|c)P(b|c)$ (e.g., Symptoms given a disease).
- Conditional independence is the "hidden sauce" of AI scalability.
- It allows us to break down massive world models.

Transition: What is Learning?

- An agent is learning if it improves behavior through experience.
- Why? Designers can't anticipate every situation.
- Why? Environments change over time.
- Learning is the bridge between a static agent and a dynamic one.

Components of a Learning Agent

- Performance Element: The part that chooses actions (The "Brain").
- Learning Element: The part that makes improvements (The "Student").
- The Critic: Observes results and provides feedback.
- Problem Generator: Suggests new experiences (Exploration).

The Role of the Critic

- The critic evaluates the agent based on a fixed standard.
- It tells the learning element: "That action was good" or "bad."
- Without a critic, the agent cannot tell if it is improving.
- Feedback is the fuel for learning.

Forms of Learning

- Supervised: Learning from labeled examples (Input/Output).
- Unsupervised: Finding patterns without a teacher.
- Reinforcement: Learning from rewards and punishments.
- Today's Focus: Supervised Learning (The foundation).

Supervised Learning: Formalism

- Given a training set of N pairs (x, y).
- x is the input; y is the output (the label).
- Find a hypothesis 'h' that approximates the true function 'f'.
- Generalization is the key—predicting 'y' for a new 'x'.

Classification vs. Regression

- Classification: Output is a discrete category (e.g., Yes/No, Cat/Dog).
- Regression: Output is a continuous number (e.g., Temperature, Price).
- We will start with Classification (Decision Trees).
- Both seek to minimize "Loss" (error in prediction).

The Hypothesis Space (H)

- H is the set of all functions the learner is allowed to consider.
- Example: The set of all possible straight lines.
- If the true function is not in H, the learner can never be perfect.
- Trade-off: Expressiveness vs. Search Complexity.

Ockham’s Razor in Learning

- "The simplest consistent hypothesis is usually the best."
- Simpler models generalize better to unseen data.
- Complexity often leads to memorizing noise (Overfitting).
- We prefer a small tree over a massive, complex one.

Consistency and Realizability

- A hypothesis is consistent if it agrees with all training data.
- A learning problem is realizable if H contains the true function.
- Real-world data is often noisy, making perfect consistency impossible.
- Goal: Find the most consistent *and* simple hypothesis.

Decision Trees: A Natural Choice

- Human-readable: A sequence of IF-THEN tests.
- Nodes = Attribute tests; Branches = Possible values.
- Leaves = Classification outcome.
- Excellent for discrete, categorical attributes.

Example: The Restaurant Wait Problem

- Goal: Predict "WillWait" for a table.
- Attributes: Patrons (None, Some, Full), WaitEstimate, Hungry.
- We have 12 training examples of past experiences.
- How do we build a tree to handle the 13th customer?

Expressiveness of Trees

- Can represent any Boolean function (DNF).
- But some functions (like Majority) require huge trees.
- Trees work best when decisions are axis-aligned.
- Our goal: Find a small tree that is consistent with the data.

Inductive Learning Strategy

- We can't check all possible trees (too many!).
- We use a "Greedy" search: Divide and Conquer.
- At each step, pick the attribute that is "most important."
- What does "important" mean mathematically?

Information Theory: Entropy

- Entropy (H) measures the "uncertainty" in the data.
- Pure set (all Yes or all No) -> H = 0.
- Totally mixed set (50/50) -> H = 1.0.
- Formula: $H(P) = -p \log_2(p) - (1-p) \log_2(1-p)$.

Information Gain (IG)

- IG = Entropy(Parent) - Weighted Remainder(Children).
- Measures how much a test "cleans up" the uncertainty.
- We split on the attribute that gives the highest Gain.
- This pushes "Yes/No" answers to the top of the tree.

Building the Tree: Step-by-Step

- 1. Calculate IG for all attributes.
- 2. Choose the winner (e.g., "Patrons").
- 3. Create branches for its values (None, Some, Full).
- 4. Recurse for each branch with the remaining data.

Handling Different Outcomes

- If all examples at a branch are same -> Done! (Leaf).
- If no attributes left but labels differ -> Majority Vote.
- If no examples left at a branch -> Plurality of Parent.
- This ensures the tree always produces an answer.

Assessing Performance: Training vs. Test

- Never test on your training data (It's cheating).
- Split data into Training Set and Test Set.
- Accuracy: % of correct predictions on the test set.
- Generalization is the only metric that matters.

The Learning Curve

- A plot of test accuracy vs. training set size.
- Accuracy starts low and rises as the agent "sees" more.
- Eventually, it plateaus as it reaches the limits of H.
- A steep curve indicates an efficient learner.

Noise and Overfitting

- Noise: Two identical inputs with different labels.
- Overfitting: Finding "patterns" in noise.
- A tree that is too deep might learn irrelevant quirks.
- Result: 100% Training accuracy but 50% Test accuracy.

Combatting Overfitting: Pruning

- Removing branches that don't help generalize.
- We look for "statistically insignificant" splits.
- If a test doesn't provide more info than a coin flip, delete it.
- Result: Simpler trees that perform better on new data.

Moving Beyond Trees: Linear Regression

- Predicting a continuous number (e.g., house price).
- Model: $y = w1*x + w0$.
- Goal: Find weights (w) that minimize squared error.
- Transition from logic-based to math-based learning.

Loss Functions

- Loss: How we quantify "being wrong."
- L2 Loss: $(y - h(x))^2$ (Penalizes big errors heavily).
- L1 Loss: $|y - h(x)|$ (Robust to outliers).
- Learning is the process of minimizing total loss.

Gradient Descent

- The engine of Machine Learning.
- "Rolling down the hill" of the loss function.
- Weights move in the direction of the steepest descent.
- Foundational for everything from Linear Regression to Deep Learning.

The Learning Rate (α)

- Controls the size of the steps in Gradient Descent.
- Too small: Training takes forever.
- Too large: You overshoot the minimum and fail.
- Tuning alpha is a core skill for AI engineers.

Logistic Regression: Soft Classification

- Using a line to separate "Spam" from "Not Spam."
- Sigmoid function: Squashes output to [0,1].
- Predicts the *probability* of an outcome.
- Bridges regression and classification.

Neural Networks: Preview

- Networks of simple linear classifiers.
- Hidden layers allow the net to "create its own features."
- Can learn complex, non-linear patterns (the "wiggly" lines).
- Next week's focus: Deep architectures.

Nonparametric Models

- Models that don't have a fixed set of weights.
- Complexity grows with the size of the data.
- Example: k-Nearest Neighbors (k-NN).
- "Show me your neighbors, and I'll tell you who you are."

k-Nearest Neighbors (k-NN)

- Classify a point by looking at the k closest training points.
- Simple, requires no "training" phase.
- But... very slow to predict in large datasets.
- Sensitive to the choice of distance metric.

Ensemble Learning: Wisdom of the Crowd

- Combining many "weak" learners into one "strong" learner.
- Random Forests: An ensemble of many randomized trees.
- Reduces overfitting and improves stability.
- One of the most effective tools in modern data science.

Practical Advice: Feature Engineering

- The model is only as good as the data you feed it.
- Turning raw data into "Attributes" is 80% of the work.
- Scaling, normalizing, and selecting features is key.
- A simple model on great features beats a complex model on bad ones.

Homework: Week 8

- 1. Calculate IG for 'Patrons' vs 'Hungry' in the restaurant set.
- 2. Draw a 2-level decision tree for a custom Boolean task.
- 3. Reading: Finish Ch. 19.3-19.5 (Decision Trees & Evaluation).

Summary: From Probability to Learning

- Naive Bayes: Fast, simple, and surprisingly robust.
- Learning: Finding a hypothesis that generalizes to unseen data.
- Decision Trees: Interpretable logic models driven by entropy.
- Overfitting: The primary danger we must control.

Next Week: Neural Networks

- Moving from "Trees" to "Nets."
- Backpropagation: How machines learn through error feedback.
- Function Approximation at scale.
- Lab: Implementation of a simple classifier in Python.